

Theoretical Foundations of Reinforcement Learning:

Week 2

Arya Patil

Algorithm Overview

Monte Carlo

Monte Carlo (MC) methods estimate action-value functions by averaging returns over complete episodes. After an episode terminates, the actual return G_t is computed backwards from the terminal state and used to update estimates:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G_t - Q(s, a)]$$

MC uses real observed returns, so estimates are unbiased. However, since a single trajectory is noisy, variance is high and many episodes are needed for convergence. MC requires episodes to terminate and cannot be applied to continuing tasks.

Q-learning

Q-learning is a model-free, off-policy algorithm that updates $Q(s, a)$ after every step using a bootstrapped target:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

The term $R + \gamma \max_{a'} Q(s', a')$ is the TD target. Q-learning does not require episode termination. It is off-policy because the update assumes a greedy target policy ($\max_{a'}$) regardless of the behaviour policy used to collect experience. Bootstrapping introduces some bias but reduces variance compared to MC.

Deep Q-Network (DQN)

DQN extends Q-learning to high-dimensional state spaces by approximating $Q(s, a)$ with a neural network. Two key stability tricks are introduced. A replay buffer stores past transitions and samples mini-batches to break temporal correlations in training data. A target network is a periodically updated copy of the main network used to compute TD

targets, preventing the target from shifting every step. DQN is limited to discrete action spaces since it requires $\max_{a'} Q(s', a')$ over all actions.

PPO

Proximal Policy Optimization is an on-policy policy gradient algorithm. Direct policy gradient methods can take update steps that are too large, destabilising the policy. PPO prevents this by clipping the policy update objective so the new policy stays close to the old one. It is stable, easy to tune, and works for both discrete and continuous action spaces. PPO is the default choice for most RL tasks.

DDPG

Deep Deterministic Policy Gradient addresses continuous action spaces where $\max_{a'} Q(s', a')$ is intractable. DDPG uses two networks: an actor that outputs a deterministic action directly, and a critic that evaluates it. Training is off-policy using a replay buffer. DDPG is sensitive to hyperparameters and can be unstable in practice.

SAC

Soft Actor-Critic extends DDPG by adding an entropy term to the objective. The agent maximises expected reward plus the entropy of its policy:

$$J(\pi) = E \left[\sum_t \gamma^t (R_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right]$$

Entropy regularisation naturally encourages exploration without requiring ε -greedy. SAC is more stable and sample-efficient than DDPG and generally preferred for continuous control tasks.

Bellman Expectation Equations — Derivation

Starting from the definition of the state-value function:

$$V_\pi(s) = E_\pi [G_t \mid S_t = s]$$

Expanding $G_t = R_{t+1} + \gamma G_{t+1}$:

$$V_\pi(s) = E_\pi [R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

Applying the law of total expectation, conditioning on the action taken and the next state:

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma E_\pi[G_{t+1} \mid S_{t+1} = s']]$$

Since $E_\pi[G_{t+1} \mid S_{t+1} = s'] = V_\pi(s')$:

$$\boxed{V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_\pi(s')]}$$

Similarly, for the action-value function $Q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$:

$$Q_\pi(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a'|s') Q_\pi(s', a') \right]$$

The two functions are related by:

$$V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a)$$

Glossary

Episode: A complete sequence of interactions from a start state to a terminal state.

Return (G_t): Total discounted future reward from timestep t : $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Discount factor (γ): Scalar in $[0, 1]$ controlling how much future rewards are down-weighted.

Policy (π): A mapping from states to actions or distributions over actions.

Value function (V_π): Expected return from state s following policy π .

Action-value function (Q_π): Expected return from state s , taking action a , then following π .

Bootstrapping: Updating estimates using other estimates rather than waiting for the true return.

TD error: Difference between the bootstrapped target and the current estimate: $R + \gamma \max_{a'} Q(s', a') - Q(s, a)$.

On-policy: The policy used to collect experience is the same as the policy being improved.

Off-policy: The policy used to collect experience differs from the policy being improved.

Replay buffer: A memory of past transitions sampled randomly during training to reduce correlation.

Target network: A periodically updated copy of the main network used to stabilise TD targets.

Entropy ($\mathcal{H}$): A measure of randomness in a distribution. Higher entropy means a more spread-out policy.

Actor-Critic: Architecture with two networks — actor outputs actions, critic evaluates them.

MDP: Markov Decision Process — the formal framework (S, A, P, R, γ) underlying RL.

Markov property: The future depends only on the current state, not the history.